

Problem Description:

1. How to check a thread is alive or not?

Solution:

Following example demonstrates how to check a thread is alive or not by extending Thread class and using `currentThread()` method.

```
public class TwoThreadAlive extends Thread {
    public void run() {
        for (int i = 0; i < 10; i++) {
            printMsg();
        }
    }

    public void printMsg() {
        Thread t = Thread.currentThread();
        String name = t.getName();
        System.out.println("name=" + name);
    }

    public static void main(String[] args) {
        TwoThreadAlive tt = new TwoThreadAlive();
        tt.setName("Thread");
        System.out.println("before start(),
        tt.isAlive()" + tt.isAlive());
        tt.start();
        System.out.println("just after start(),
        tt.isAlive()" + tt.isAlive());
        for (int i = 0; i < 10; i++) {
            tt.printMsg();
        }
        System.out.println("The end of main(),
        tt.isAlive()" + tt.isAlive());
    }
}
```

Result:

The above code sample will produce the following result.

```
before start(),tt.isAlive()=false
just after start(), tt.isAlive()=true
name=main
name=main
name=main
name=main
name=main
name=main
name=Thread
name=Thread
name=Thread
name=Thread
name=Thread
name=Thread
name=Thread
name=main
name=main
The end of main(),tt.isAlive()=false
```

Problem Description:

2. How to check a thread has stopped or not?

Solution:

Following example demonstrates how to check a thread has stop or not by checking with `isAlive()` method.

```
public class Main {
    public static void main(String[] argv)
        throws Exception {
        Thread thread = new MyThread();
        thread.start();
        if (thread.isAlive()) {
            System.out.println("Thread has not finished");
        }
        else {
            System.out.println("Finished");
        }
        long delayMillis = 5000;
        thread.join(delayMillis);
        if (thread.isAlive()) {
            System.out.println("thread has not finished");
        }
        else {
            System.out.println("Finished");
        }
        thread.join();
    }
}

class MyThread extends Thread {
    boolean stop = false;
    public void run() {
        while (true) {
            if (stop) {
                return;
            }
        }
    }
}
```

Result:

The above code sample will produce the following result.

```
Thread has not finished
Finished
```

Problem Description:

4. How to get the priorities of running threads?

Solution:

Following example prints the priority of the running threads by using `setPriority()` method .

```
public class SimplePriorities extends Thread {
    private int countDown = 5;
    private volatile double d = 0;
    public SimplePriorities(int priority) {
        setPriority(priority);
        start();
    }
    public String toString() {
        return super.toString() + ": " + countDown;
    }
    public void run() {
        while(true) {
            for(int i = 1; i < 100000; i++)
                d = d + (Math.PI + Math.E) / (double)i;
            System.out.println(this);
            if(--countDown == 0) return;
        }
    }
    public static void main(String[] args) {
        new SimplePriorities(Thread.MAX_PRIORITY);
        for(int i = 0; i < 5; i++)
            new SimplePriorities(Thread.MIN_PRIORITY);
    }
}
```

Result:

The above code sample will produce the following result.

```
Thread[Thread-0,10,main]: 5
Thread[Thread-0,10,main]: 4
Thread[Thread-0,10,main]: 3
Thread[Thread-0,10,main]: 2
Thread[Thread-0,10,main]: 1
Thread[Thread-1,1,main]: 5
Thread[Thread-1,1,main]: 4
Thread[Thread-1,1,main]: 3
Thread[Thread-1,1,main]: 2
Thread[Thread-1,1,main]: 1
Thread[Thread-2,1,main]: 5
Thread[Thread-2,1,main]: 4
Thread[Thread-2,1,main]: 3
Thread[Thread-2,1,main]: 2
Thread[Thread-2,1,main]: 1
.
.
.
```

Problem Description:

6. How to get the name of a running thread?

Solution:

Following example shows How to get the name of a running thread .

```
public class TwoThreadGetName extends Thread {
    public void run() {
        for (int i = 0; i < 10; i++) {
            printMsg();
        }
    }
    public void printMsg() {
        Thread t = Thread.currentThread();
        String name = t.getName();
        System.out.println("name=" + name);
    }
    public static void main(String[] args) {
        TwoThreadGetName tt = new TwoThreadGetName();
        tt.start();
        for (int i = 0; i < 10; i++) {
            tt.printMsg();
        }
    }
}
```

Result:

The above code sample will produce the following result.

```
name=main
name=main
name=main
name=main
name=main
name=thread
name=thread
name=thread
name=thread
```

Problem Description:

8. How to set the priority of a thread?

Solution:

Following example how to set the priority of a thread with the help of.

```
public class Main {  
    public static void main(String[] args)  
        throws Exception {  
        Thread thread1 = new Thread(new TestThread(1));  
        Thread thread2 = new Thread(new TestThread(2));  
        thread1.setPriority(Thread.MAX_PRIORITY);  
        thread2.setPriority(Thread.MIN_PRIORITY);  
        thread1.start();  
        thread2.start();  
        thread1.join();  
        thread2.join();  
        System.out.println("The priority has been set.");  
    }  
}
```

Result:

The above code sample will produce the following result.

```
The priority has been set.
```

Problem Description:

9. How to stop a thread for a while?

Solution:

Following example demonstrates how to stop a thread by creating an user defined method run() taking the help of Timer classes' methods.

```
import java.util.Timer;
import java.util.TimerTask;

class CanStop extends Thread {
    private volatile boolean stop = false;
    private int counter = 0;
    public void run() {
        while (!stop && counter < 10000) {
            System.out.println(counter++);
        }
        if (stop)
            System.out.println("Detected stop");
    }
    public void requestStop() {
        stop = true;
    }
}

public class Stopping {
    public static void main(String[] args) {
        final CanStop stoppable = new CanStop();
        stoppable.start();
        new Timer(true).schedule(new TimerTask() {
            public void run() {
                System.out.println("Requesting stop");
                stoppable.requestStop();
            }
        }, 350);
    }
}
```

Result:

The above code sample will produce the following result.

Detected stop

Problem Description:

10. How to suspend a thread for a while?

Solution:

Following example shows how to suspend a thread for a while by creating sleepingThread() method.

```
public class SleepingThread extends Thread {
    private int countDown = 5;
    private static int threadCount = 0;
    public SleepingThread() {
        super("" + ++threadCount);
        start();
    }
    public String toString() {
        return "#" + getName() + ": " + countDown;
    }
    public void run() {
        while (true) {
            System.out.println(this);
            if (--countDown == 0)
                return;
            try {
                sleep(100);
            }
            catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
    }
    public static void main(String[] args)
        throws InterruptedException {
        for (int i = 0; i < 5; i++)
            new SleepingThread().join();
        System.out.println("The thread has been suspended.");
    }
}
```

Result:

The above code sample will produce the following result.

```
The thread has been suspended.
```

Problem Description:

12. How to check priority level of a thread?

Solution:

Following example demonstrates how to check priority level of a thread using `getPriority()` method of a `Thread`.

```
public class Main extends Object {
    private static Runnable makeRunnable() {
        Runnable r = new Runnable() {
            public void run() {
                for (int i = 0; i < 5; i++) {
                    Thread t = Thread.currentThread();
                    System.out.println("in run() - priority="
                        + t.getPriority() + ", name=" + t.getName());
                    try {
                        Thread.sleep(2000);
                    }
                    catch (InterruptedException x) {}
                }
            }
        };
        return r;
    }
    public static void main(String[] args) {
        System.out.println("in main() - Thread.currentThread().
            getPriority()=" + Thread.currentThread().getPriority());
        System.out.println("in main() - Thread.currentThread()
            .getName()=" + Thread.currentThread().getName());
        Thread threadA = new Thread(makeRunnable(), "threadA");
        threadA.start();
        try {
            Thread.sleep(3000);
        }
        catch (InterruptedException x) {}
        System.out.println("in main() - threadA.getPriority()="
            + threadA.getPriority());
    }
}
```

Result:

The above code sample will produce the following result.

```
in main() - Thread.currentThread().getPriority()=5
in main() - Thread.currentThread().getName()=main
in run() - priority=5, name=threadA
in run() - priority=5, name=threadA
in main() - threadA.getPriority()=5
in run() - priority=5, name=threadA
in run() - priority=5, name=threadA
in run() - priority=5, name=threadA
```

Problem Description:

13. How to display all running Thread?

Solution:

Following example demonstrates how to display names of all the running threads using getName() method.

```
public class Main extends Thread {
    public static void main(String[] args) {
        Main t1 = new Main();
        t1.setName("thread1");
        t1.start();
        ThreadGroup currentGroup =
            Thread.currentThread().getThreadGroup();
        int noThreads = currentGroup.activeCount();
        Thread[] lstThreads = new Thread[noThreads];
        currentGroup.enumerate(lstThreads);
        for (int i = 0; i < noThreads; i++)
            System.out.println("Thread No:" + i + " = "
                + lstThreads[i].getName());
    }
}
```

Result:

The above code sample will produce the following result.

```
Thread No:0 = main
Thread No:1 = thread1
```